

Накарон — Архитектура решения

Содержание

- [Обзор](#обзор)
- [Бизнес-логика и флоу](#бизнес-логика-и-флоу)
- [Стек технологий](#стек-технологий)
- [Архитектура системы](#архитектура-системы)
- [Docker-инфраструктура](#docker-инфраструктура)
- [Роли пользователей и интерфейсы](#роли-пользователей-и-интерфейсы)
- [Схема базы данных](#схема-базы-данных)
- [API-эндпоинты](#api-эндпоинты)
- [Аутентификация и авторизация](#аутентификация-и-авторизация)
- [Frontend-архитектура](#frontend-архитектура)
- [ML-сервис](#ml-сервис)
- [Структура проекта](#структура-проекта)

Обзор

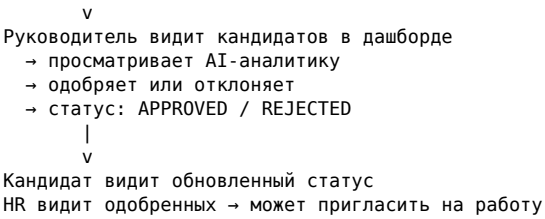
Накарон — AI-платформа для интеллектуального отбора кандидатов. Платформа автоматизирует полный цикл рекрутинга: от создания вакансии с вопросами до финального одобрения кандидата руководителем.

Ключевая идея: кандидат отвечает на набор вопросов (системных + кастомных), AI анализирует ответы и формирует детальную оценку, HR фильтрует лучших и отправляет руководителю на финальное решение.

Бизнес-логика и флоу

Основной поток

```
HR создает вакансию
+ выбирает вопросы из банка (15 системных + свои)
|
v
Кандидат регистрируется (email + phone + пароль)
→ видит список вакансий
→ выбирает вакансию
→ проходит анкету (отвечает на все вопросы)
|
v
AI анализирует ответы (mock / ML-сервис)
→ score/100, vacancy_match%, потенциал роста
→ сильные стороны, зоны развития, AI-резюме
→ статус: ANALYZED
|
v
HR видит результаты AI в разделе "Отчёты"
→ просматривает анализ
→ открывает Досье (ответы кандидата + профиль)
→ отправляет лучших руководителю
→ статус: SENT_TO_MANAGER
|
```



Жизненный цикл кандидата (статусы)

PENDING → PROCESSING → ANALYZED → HR_REVIEW → SENT_TO_MANAGER → APPROVED
 → REJECTED

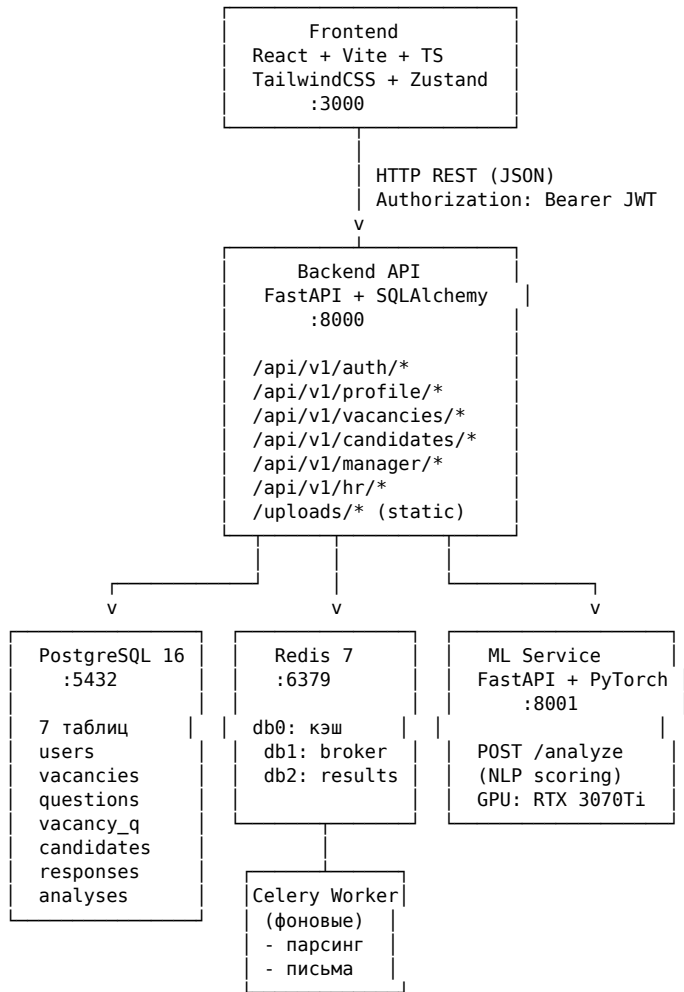
Статус	Кто видит	Описание
PENDING	Система	Ответы получены, анализ не начат
PROCESSING	Система	ML-сервис обрабатывает
ANALYZED	HR	Анализ завершён, ждёт HR-ревью
HR_REVIEW	HR	HR изучает кандидата
SENT_TO_MANAGER	Руководитель	HR отправил на финальное решение
APPROVED	Все	Руководитель одобрил
REJECTED	Все	Руководитель отклонил

Стек технологий

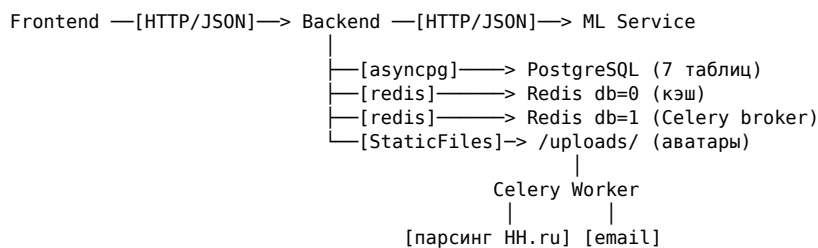
Слой	Технологии
Frontend	React 18, TypeScript, Vite, TailwindCSS, Zustand, React Router v6, Axios
Backend API	Python 3.11+, FastAPI, SQLAlchemy 2.0 (async), Pydantic v2
ML Service	Python 3.11+, FastAPI, PyTorch, Transformers, scikit-learn
База данных	PostgreSQL 16
Кэш / Брокер	Redis 7 (кэш db=0, Celery broker db=1, Celery results db=2)
Фоновые задачи	Celery
Аутентификация	JWT (access 30min + refresh 7d), bcrypt, python-jose
Файлы	StaticFiles (аватары → /uploads/avatars/)
Контейнеризация	Docker, Docker Compose

Архитектура системы

Диаграмма сервисов



Связи между сервисами



Docker-инфраструктура

Проект запускается через `docker-compose.yml` (production) + `docker-compose.dev.yml` (dev с hot-reload).

Сервис	Образ	Порт	Зависимости	Healthcheck
postgres	postgres:16-alpine	5432	—	pg_isready
redis	redis:7-alpine	6379	—	redis-cli ping
backend	./backend/Dockerfile	8000	postgres, redis	—
ml-service	./ml-service/Dockerfile	8001	—	—

celery-worker	./backend/Dockerfile	—	postgres, redis	—
frontend	./frontend/Dockerfile	3000	backend	—

Запуск (dev):

```
docker-compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
```

Volumes: postgres_data, redis_data

Роли пользователей и интерфейсы

Кандидат (candidate)

Создаётся автоматически при регистрации. Не может выбрать другую роль.

Страница	Маршрут	Описание
Вакансии	/vacancies	Список активных вакансий
Анкета	/questionnaire/:vacancyId	Пошаговая форма с вопросами
Мой статус	/status	Статус рассмотрения заявки
Профиль	/profile	Редактирование профиля, аватар

HR (hr)

Создаётся вручную в БД (UPDATE role = 'HR'). Управляет вакансиями и фильтрует кандидатов.

Страница	Маршрут	Описание
Вакансии	/hr	Список вакансий, создание новых
Создание вакансии	/hr/vacancies/create	Форма + выбор вопросов из банка
Детали вакансии	/hr/vacancies/:id	Вакансия + кандидаты
Ревью кандидата	/hr/candidates/:id	AI-анализ + отправка руководителю
Отчёты	/hr/reports	Вакансии с количеством кандидатов
Отчёт по вакансии	/hr/reports/vacancy/:id	Все кандидаты по вакансии (все статусы)
Анализ кандидата	/hr/reports/candidate/:id	AI-аналитика + кнопка "Досье"
Досье кандидата	/hr/reports/candidate/:id/dossier	Профиль + AI-оценка + ВСЕ ответы
Одобрённые	/hr/approved	Кандидаты после одобрения руководителем
Профиль	/profile	Редактирование своего профиля

Руководитель (manager)

Создаётся вручную в БД (UPDATE role = 'MANAGER'). Принимает финальное решение.

Страница	Маршрут	Описание
----------	---------	----------

Дашборд	/manager	Кандидаты, отправленные HR (SENT_TO_MANAGER)
Детали кандидата	/manager/candidate/:id	AI-аналитика + одобрить/отклонить
Профиль	/profile	Редактирование своего профиля

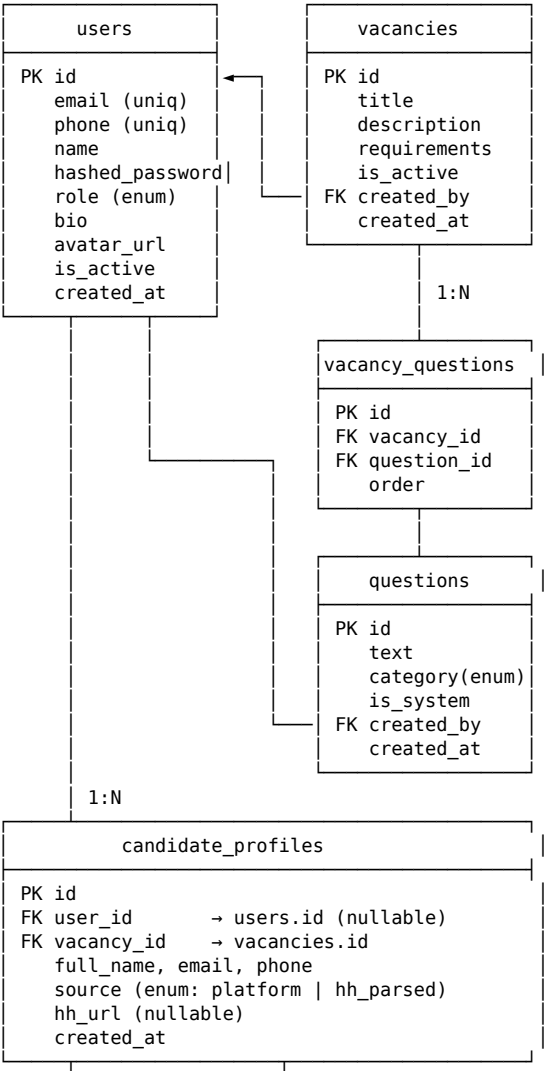
Навигация в header

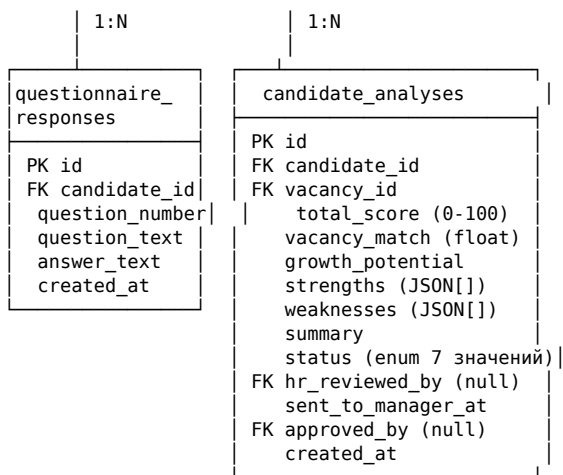
Роль	Пункты меню
Кандидат	Вакансии, Мой статус
HR	Вакансии, Отчёты, Одобренные
Руководитель	Дашборд

Все роли: клик по имени/аватару → Профиль, кнопка "Выйти".

Схема базы данных

ER-диаграмма





Таблицы

users

Поле	Тип	Ограничения
id	INTEGER	PK, autoincrement
email	VARCHAR(255)	UNIQUE, INDEX
phone	VARCHAR(50)	UNIQUE, INDEX
name	VARCHAR(255)	
hashed_password	VARCHAR(255)	
role	ENUM	candidate / manager / hr
bio	TEXT	default: ""
avatar_url	VARCHAR(500)	nullable
is_active	BOOLEAN	default: true
created_at	TIMESTAMPTZ	default: now()

vacancies

Поле	Тип	Ограничения
id	INTEGER	PK
title	VARCHAR(255)	
description	TEXT	
requirements	TEXT	default: ""
is_active	BOOLEAN	default: true
created_by	INTEGER	FK → users.id
created_at	TIMESTAMPTZ	default: now()

questions (банк вопросов)

Поле	Тип	Ограничения
id	INTEGER	PK
text	TEXT	

category	ENUM	experience / competencies / motivation / potential
is_system	BOOLEAN	default: false
created_by	INTEGER	FK → users.id, nullable (null для системных)
created_at	TIMESTAMPZ	default: now()

15 системных вопросов загружаются автоматически при старте бекенда (seed).

vacancy_questions (M2M: вакансии ↔ вопрос)

Поле	Тип	Ограничения
id	INTEGER	PK
vacancy_id	INTEGER	FK → vacancies.id
question_id	INTEGER	FK → questions.id
order	INTEGER	default: 0

candidate_profiles

Поле	Тип	Ограничения
id	INTEGER	PK
user_id	INTEGER	FK → users.id, nullable
vacancy_id	INTEGER	FK → vacancies.id
full_name	VARCHAR(255)	
email	VARCHAR(255)	
phone	VARCHAR(50)	default: ""
source	ENUM	platform / hh_parsed
hh_url	VARCHAR(500)	nullable
created_at	TIMESTAMPZ	default: now()

questionnaire_responses

Поле	Тип	Ограничения
id	INTEGER	PK
candidate_id	INTEGER	FK → candidate_profiles.id
question_number	INTEGER	
question_text	TEXT	
answer_text	TEXT	
created_at	TIMESTAMPZ	default: now()

candidate_analyses

Поле	Тип	Ограничения
id	INTEGER	PK
candidate_id	INTEGER	FK → candidate_profiles.id
vacancy_id	INTEGER	FK → vacancies.id
total_score	INTEGER	default: 0

vacancy_match	FLOAT	default: 0.0
growth_potential	TEXT	default: ""
strengths	JSON	default: []
weaknesses	JSON	default: []
summary	TEXT	default: ""
status	ENUM	7 значений (см. выше)
hr_reviewed_by	INTEGER	FK → users.id, nullable
sent_to_manager_at	TIMESTAMPZ	nullable
approved_by	INTEGER	FK → users.id, nullable
created_at	TIMESTAMPZ	default: now()

API-эндпоинты

Все API под префиксом `/api/v1/`.

Auth (/auth)

Метод	Путь	Доступ	Описание
POST	/auth/register	Public	Регистрация (email, phone, name, password) → JWT
POST	/auth/login	Public	Вход (email, password) → JWT
GET	/auth/me	JWT	Текущий пользователь

Profile (/profile)

Метод	Путь	Доступ	Описание
GET	/profile/	JWT	Получить свой профиль
PUT	/profile/	JWT	Обновить имя, телефон, bio
POST	/profile/avatar	JWT	Загрузить аватар (multipart, max 5MB)

Vacancies (/vacancies)

Метод	Путь	Доступ	Описание
GET	/vacancies/	Public	Список активных вакансий
GET	/vacancies/{id}	Public	Детали вакансии с вопросами

Candidates (/candidates)

Метод	Путь	Доступ	Описание
-------	------	--------	----------

POST	/candidates/submit-questionnaire	Candidate	Отправить анкету → создать profile + answers + mock AI analysis
GET	/candidates/my-status	Candidate	Статус своей последней заявки

Manager (/manager)

Метод	Путь	Доступ	Описание
GET	/manager/candidates	Manager	Кандидаты со статусом SENT_TO_MANAGER
GET	/manager/candidates/{id}/analysis	Manager	Полный AI-анализ
POST	/manager/candidates/{id}/approve	Manager	Одобрить → APPROVED
POST	/manager/candidates/{id}/reject	Manager	Отклонить → REJECTED

HR (/hr)

Метод	Путь	Доступ	Описание
GET	/hr/vacancies	HR	Все вакансии
POST	/hr/vacancies	HR	Создать вакансию
GET	/hr/vacancies/{id}	HR	Детали вакансии с вопросами
GET	/hr/vacancies/{id}/candidates	HR	Все кандидаты по вакансии (все статусы)
GET	/hr/questions/bank	HR	Банк вопросов (системные + кастомные)
POST	/hr/questions	HR	Создать кастомный вопрос
POST	/hr/vacancies/{id}/questions	HR	Привязать вопросы к вакансии
GET	/hr/candidates	HR	Кандидаты со статусом ANALYZED
GET	/hr/candidates/{id}	HR	Полный AI-анализ кандидата
GET	/hr/candidates/{id}/dossier	HR	Досье: профиль + AI + ответы
POST	/hr/candidates/{id}/send-to-manager	HR	Отправить руководителю
GET	/hr/approved	HR	Одобренные кандидаты
POST	/hr/candidates/{id}/invite	HR	Пригласить кандидата

Health

Метод	Путь	Доступ	Описание
GET	/health	Public	Backend health check

ML Service (порт 8001):

Метод	Путь	Описание
POST	/api/v1/analyze	AI-анализ ответов кандидата
GET	/health	ML service health check

Аутентификация и авторизация

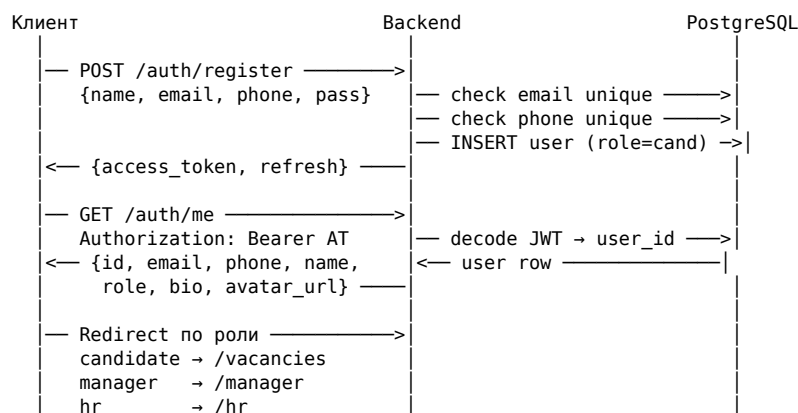
Механизм

- Пароли: bcrypt (passlib + bcrypt 4.0.1)
- Токены: JWT (python-jose), алгоритм HS256
- Access token: 30 минут
- Refresh token: 7 дней
- Хранение на клиенте: localStorage.access_token

Регистрация

- Роль всегда CANDIDATE (зашиито на бекенде)
- Валидация: уникальность email + уникальность phone
- HR и Manager создаются вручную через БД

Поток



Role-based access (frontend)

- ProtectedRoute компонент проверяет isAuthenticated + allowedRoles
- Неавторизован → /auth
- Чужая роль → редирект на дефолтную страницу своей роли

Frontend-архитектура

State Management

- Zustand (authStore) — пользователь, JWT, login/register/logout/hydrate
- Гидратация при загрузке: проверка токена через GET /me

Роутинг

- React Router v6 — вложенные маршруты через +
- RootRedirect — при входе на / редирект по роли

API-клиент

- Axios с interceptors:
- Request: подставляет Authorization: Bearer из localStorage
- Response: при 401 → очистка токена → редирект на /auth

Страницы (16)

Страница	Файл	Описание
AuthPage	AuthPage.tsx	Табы Вход/Регистрация
ProfilePage	ProfilePage.tsx	Аватар, имя, телефон, bio
VacanciesPage	VacanciesPage.tsx	Список вакансий (для кандидата)
CandidateQuestionnairePage	CandidateQuestionnairePage.tsx	Пошаговая анкета
CandidateStatusPage	CandidateStatusPage.tsx	Статус заявки
ManagerDashboardPage	ManagerDashboardPage.tsx	Кандидаты SENT_TO_MANAGER
ManagerCandidateDetailPage	ManagerCandidateDetailPage.tsx	AI-анализ + одобрить/отклонить
HrDashboardPage	HrDashboardPage.tsx	Вакансии + кандидаты после AI
HrVacancyCreatePage	HrVacancyCreatePage.tsx	Создание вакансии + выбор вопросов
HrVacancyDetailPage	HrVacancyDetailPage.tsx	Вакансия + вопросы + кандидаты
HrCandidateReviewPage	HrCandidateReviewPage.tsx	AI-анализ + отправка руководителю
HrReportsPage	HrReportsPage.tsx	Отчёты: вакансии + кол-во кандидатов
HrReportVacancyCandidatesPage	HrReportVacancyCandidatesPage.tsx	Кандидаты по вакансии (все статусы)
HrReportCandidateAnalysisPage	HrReportCandidateAnalysisPage.tsx	AI-анализ + кнопка "Досье"
HrCandidateDossierPage	HrCandidateDossierPage.tsx	Досье: профиль + AI + ответы
HrApprovedPage	HrApprovedPage.tsx	Одобренные → приглашение

ML-сервис

Текущее состояние

- Отдельный FastAPI на порту 8001
- Эндпоинт POST /api/v1/analyze — возвращает mock-данные
- AI-анализ пока эмулируется на бекенде (random scores при submit-questionnaire)

Целевая архитектура

- GPU: NVIDIA RTX 3070 Ti (8GB VRAM)

- Рекомендуемые модели:
- cointegrated/rubert-tiny2 (~120MB) — быстрый
- ai-forever/sbert_large_nlu_ru (~1.3GB) — качественный
- sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (~470MB) — баланс
- Pipeline: preprocessing → embeddings → scoring → vacancy matching → summary generation

Банк системных вопросов (15 шт.)

#	Категория	Вопрос (сокращённо)
1-4	Опыт	Достижения, проблемы, неудачи, сложные задачи
5-8	Компетенции	Навыки, обучаемость, командная работа, новые задачи
9-12	Мотивация	Интерес к вакансии, цели, ценности, инициатива
13-15	Потенциал	Свобода проекта, реакция на изменения, развитие

Структура проекта

```
Hakaron/
├── docker-compose.yml           # Production compose
├── docker-compose.dev.yml      # Dev override (hot-reload)
├── .env                        # Переменные окружения
├── .gitignore
├── Readme.txt
├── frontend/                  # React SPA
│   ├── Dockerfile / Dockerfile.dev
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   ├── index.html
│   └── src/
│       ├── main.tsx           # Entry point
│       ├── App.tsx            # Роутинг (22 маршрута)
│       ├── index.css           # Tailwind imports
│       ├── api/client.ts       # Axios + JWT interceptors
│       ├── store/authStore.ts  # Zustand auth state
│       ├── types/index.ts      # TypeScript интерфейсы
│       ├── components/
│       │   └── ProtectedRoute.tsx # Role-based route guard
│       ├── layouts/
│       │   └── MainLayout.tsx   # Header + navigation + Outlet
│       └── pages/              # 16 страниц
│           ├── AuthPage.tsx
│           ├── ProfilePage.tsx
│           ├── VacanciesPage.tsx
│           ├── CandidateQuestionnairePage.tsx
│           ├── CandidateStatusPage.tsx
│           ├── ManagerDashboardPage.tsx
│           ├── ManagerCandidateDetailPage.tsx
│           ├── HrDashboardPage.tsx
│           ├── HrVacancyCreatePage.tsx
│           ├── HrVacancyDetailPage.tsx
│           ├── HrCandidateReviewPage.tsx
│           ├── HrReportsPage.tsx
│           ├── HrReportVacancyCandidatesPage.tsx
│           ├── HrReportCandidateAnalysisPage.tsx
│           ├── HrCandidateDossierPage.tsx
│           └── HrApprovedPage.tsx
└── backend/                   # FastAPI API
    ├── Dockerfile / Dockerfile.dev
    ├── requirements.txt
    └── app/
```

```

├── main.py # FastAPI app + lifespan + seed
├── core/
│   ├── config.py # Pydantic Settings
│   ├── database.py # SQLAlchemy async engine
│   └── security.py # JWT + bcrypt
├── models/ # 7 SQLAlchemy моделей
│   ├── user.py
│   ├── vacancy.py
│   ├── question.py
│   ├── vacancy_question.py
│   ├── candidate.py
│   ├── questionnaire.py
│   └── analysis.py
├── schemas/ # Pydantic DTOs
│   ├── auth.py
│   ├── profile.py
│   ├── vacancy.py
│   ├── question.py
│   └── candidate.py
├── api/v1/ # 6 роутеров, 25+ эндпоинтов
│   ├── auth.py
│   ├── profile.py
│   ├── vacancies.py
│   ├── candidates.py
│   ├── manager.py
│   └── hr.py
├── services/
│   ├── analysis_service.py # HTTP клиент к ML
│   └── parser_service.py # HH.ru API
├── tasks/ # Celery
│   ├── parsing.py
│   └── notifications.py
├── ml-service/ # ML FastAPI
│   ├── Dockerfile / Dockerfile.dev
│   ├── requirements.txt
│   └── app/
│       ├── main.py
│       ├── api/analyze.py # POST /api/v1/analyze
│       └── services/
│           └── nlp_analyzer.py # Placeholder для ML pipeline
├── docs/ # Документация
│   ├── ARCHITECTURE.md # Этот документ
│   ├── FRONTEND_SPEC.md
│   ├── BACKEND_SPEC.md
│   ├── ML_SPEC.md
│   └── API_SPEC.md

```